Space Race Analysis

A Deep Dive

Portfolio explainer, written for a reader with no prior knowledge

The one-paragraph version

This project takes a single messy spreadsheet of 4,324 rocket launches (1957 to 2020, scraped from a spaceflight news site) and turns it into charts. It has two front ends over the same data: a linear script, `space_race_analysis.py`, that writes 17 chart files into `output/` and exits; and a small Flask web dashboard, `app.py`, that serves five charts you can filter live in a browser. The cleaning logic is short: throw away two junk columns, map each launch's location text to a country code, and pull a year out of the date. Everything else is chart-drawing. The analysis is real and the numbers in the README are all accurate, but several of the charts encode something other than what their titles claim, and the project's marquee result, the "USA vs USSR" comparison, rests on a substitution that understates the USA by roughly a factor of four. All of that is documented in Section **??**.

Contents

1 Orientation

1.1 What this project is, and what it is not

It is a *descriptive* data analysis. It answers "what happened" questions (who launched the most, how often launches failed, what a launch cost) by counting and averaging rows in a table and drawing the result. It does not predict anything, it does not fit a statistical model, and it does not test a hypothesis. There is no machine learning here and none is claimed.

It is also, structurally, two programs that happen to share one CSV file and one cleaning recipe. Understanding that split is most of understanding the repository.

1.2 Vocabulary you need before anything else

Every term below is used constantly in the rest of this document.

Jargon: CSV

"Comma-Separated Values". A plain text file that stores a table. The first line usually names the columns, and every line after it is one row, with commas between the cells. It is the lowest common denominator of data exchange: any spreadsheet or database can read and write it. `mission_launches.csv` in this repository is one, and it holds 4,324 launch records on 4,325 lines (one header line plus one line per launch).

Jargon: pandas and the DataFrame

`pandas` is the standard Python library for working with tables. Its central object is the *DataFrame*, which you can picture as a spreadsheet held in memory: named columns, numbered rows, and a large vocabulary of operations that act on a whole column at once rather than one cell at a time. When this document says "the dataframe", it means the in-memory table loaded from the CSV. A single column pulled out of a DataFrame is called a *Series*.

Jargon: NaN and "missing"

`NaN` means "Not a Number". pandas uses it as a universal marker for a cell that has no value: the source data never recorded it. It is contagious in arithmetic (anything plus `NaN` is `NaN`), which is why code that cares about correctness must decide explicitly whether to drop missing rows or fill them in. This project drops them, which is the right call and is discussed in Section 6.5.

Jargon: matplotlib and Plotly

Two Python charting libraries with different outputs. `matplotlib` draws a static picture and saves it as a PNG (an image file: pixels, no interactivity). `Plotly` builds a chart as a data structure that a JavaScript library renders in a web browser, so the result can be hovered, zoomed and clicked. This project uses matplotlib for 15 charts and Plotly for 2 in the static script, and Plotly for all 5 in the dashboard.

Jargon: Flask

A small Python *web framework*: a library that listens for HTTP requests from a browser and lets you write a Python function for each URL. "Small" is the point; it supplies routing and templating and leaves everything else to you. `app.py` uses it to serve one page and one JSON endpoint.

Jargon: ISO alpha-3 country code

A three-letter code standardised by the International Organization for Standardization that uniquely names a country: `USA`, `RUS`, `KAZ`, `FRA`. Mapping software prefers codes over names because names are ambiguous and spelled inconsistently, while codes are not. Plotly's map charts expect them.

2 The dataset

2.1 Provenance

`mission_launches.csv` was scraped from `nextspaceflight.com` and ships inside the repository. The README is explicit that regenerating it would require re-scraping the source site, so the file is treated as the fixed input, not as something the project produces. It is small (a few hundred kilobytes), which is why committing it is reasonable rather than sloppy.

2.2 The columns

After cleaning, seven columns survive. Three more are derived by the code.

Column	Origin	Meaning
`Organisation`	source	Who launched it. 56 distinct values, e.g. `RVSN USSR`, `NASA`, `SpaceX`.
`Location`	source	Free text, roughly "pad, site, region, country".
`Date`	source	Launch date and time as text, in more than one format.
`Detail`	source	Rocket and payload, e.g. `Falcon 9 Block 5 | Starlink V1 L9`. Never analysed.
`Rocket_Status`	source	`StatusActive` or `StatusRetired`.
`Price`	source	Cost in millions of USD. Present on only 964 of 4,324 rows.
`Mission_Status`	source	`Success`, `Failure`, `Partial Failure` or `Prelaunch Failure`.
`Country`	derived	ISO alpha-3 code parsed out of `Location`.
`Year`	derived	Integer year parsed out of `Date`.
`Month`	derived	Month name parsed out of `Date`.

Table 1: The dataframe's columns after cleaning. `Detail` is loaded and never used.

2.3 The two junk columns, and why there are exactly two

Both the static script and the shared loader run this line:

```
df.drop(columns=['Unnamed: 0', 'Unnamed: 0.1'], inplace=True)
```

The README calls these "two junk index columns scraped in by mistake", which is true but skips the interesting part: *why they have those particular names*. The first line of the CSV is:

```
,Unnamed: 0,Organisation,Location,Date,Detail,Rocket_Status,Price,Mission_Status
```

Read it carefully. The first field is *empty*, and the second field is *literally the text* `Unnamed: 0`. That is a fossil of the data being saved twice: some earlier tool wrote its row numbers out as an unnamed column, something read that back and named the nameless column `Unnamed: 0`, and then wrote it out *again* with a fresh unnamed row-number column in front. So the file carries two generations of the same mistake.

When pandas reads this, it must invent a name for the empty first field, and its convention is exactly `Unnamed: 0`. That collides with the real column already called `Unnamed: 0`, so pandas de-duplicates by appending a suffix. The observed result, confirmed by loading the file, is that the columns arrive in this order:

```
['Unnamed: 0.1', 'Unnamed: 0', 'Organisation', 'Location', 'Date',
 'Detail', 'Rocket_Status', 'Price', 'Mission_Status']
```

The blank-headed first column became `Unnamed: 0.1`; the literal one kept `Unnamed: 0`. The drop removes both, so the outcome is correct either way, but the names are a pandas artefact rather than anything in the file, and that is worth being able to explain.

This line is load-bearing and silently version-dependent

The de-duplication suffix (`.1`) is a pandas naming convention, not a standard. The drop is hardcoded to both literal names, so a pandas release that mangled duplicates differently, or a re-scrape that emitted one artefact column instead of two, would raise `KeyError` on this line and stop the program. That failure would at least be loud, which is better than the silent alternatives elsewhere in this codebase, but it is a hardcoded assumption about a file the project does not control. Passing `errors='ignore'`, or dropping by a pattern match on the prefix, would be sturdier.

2.4 The data is not as clean as the README believes

The README's "What I'd do differently" section says the price cast works "because the CSV happens to be clean". Inspecting the file directly shows it is not.

- One organisation name is stored as `Arm??e de l'Air` (the French air force, correctly *Armée de l'Air*). Those are two literal ASCII question marks sitting in the data, and they appear on the axis of any chart that shows that organisation. There is no accent to recover: the information is gone, baked in at scrape time.
- 16 `Location` values and 325 `Detail` values contain broken multi-byte sequences. One `Location` reads as `M`, then an invalid byte, then `hia Peninsula, New Zealand`; it should read *Māhia Peninsula*. Another `Detail` value renders as `Pics Or It Didn`, then two corrupt bytes, then `t Happen`, where the original had an apostrophe.

Mojibake in the source data, undocumented

This is classic *mojibake*: text encoded in one character set and decoded as another, scrambling every non-English character. It happened upstream in the scrape and the project inherits it. It is mostly harmless here because the two corrupted columns are cosmetic (`Detail` is never analysed at all, and `Location` is only ever split on its last comma, which the corruption does not touch). But the README asserts the opposite, that the CSV is clean, and an interviewer who greps the file for non-ASCII characters will find this in about five seconds. Better to say plainly: the data has known encoding damage in `Detail` and `Location`, it does not affect the analysis, and here is why.

Note for anyone regenerating this document: those corrupt bytes must never be pasted into a LaTeX listing. `pdflatex` has no font for them and the build fails hard on the first invalid byte, so they are described here rather than reproduced.

3 Repository layout

```
space-race-analysis
├── space_race_analysis.py,   the static script: load, clean, 17 charts, exit
├── app.py,   the Flask dashboard: 5 live charts
├── data_loader.py,   shared load+clean, used only by app.py
├── templates/
│   └── index.html,   the dashboard page: CSS, markup, client JS
├── mission_launches.csv,   the input. 4,324 launches
├── output/,   15 PNGs + 2 HTMLs, one committed sample run
├── requirements.txt
├── README.md
└── LICENSE
```

Figure 1: The whole repository. Ten tracked items plus the output folder. There is no package, no test suite, and no configuration file.

No tests exist

There is no test file and no test framework in `requirements.txt`. For a linear analysis script this is a defensible trade, and the README's own "What I'd do differently" proposes adding assertions after the cleaning step. But it should be stated as a choice, not left to be discovered: nothing in this repository can detect a regression automatically. The verification in Section 9 was all done by hand.

4 Architecture

The shape of the project is one input feeding two independent consumers that never talk to each other.

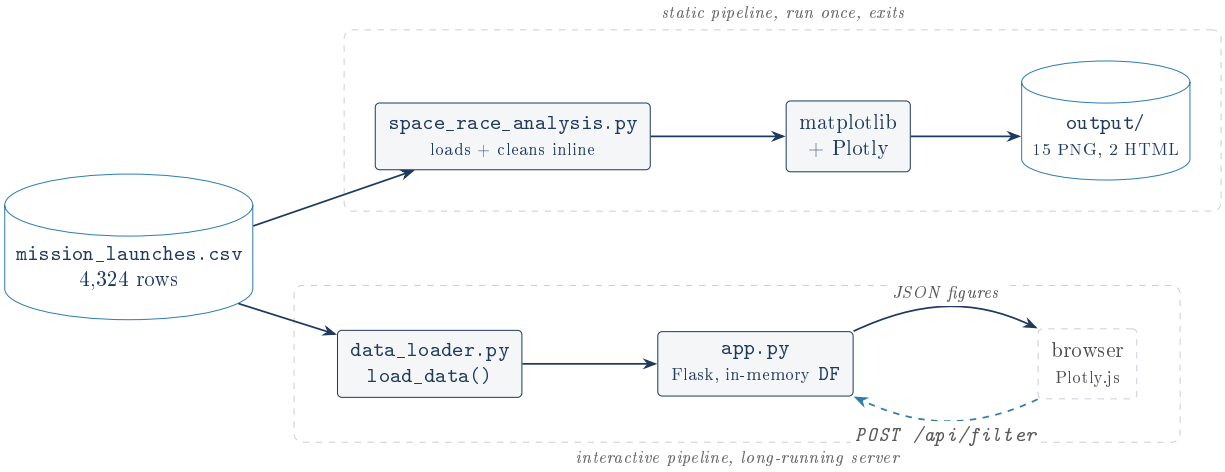

Figure 2: Component map. The two pipelines share the CSV and the cleaning *recipe*, but share no code: `space_race_analysis.py` does not import `data_loader.py`.

The cleaning logic is duplicated, deliberately, and that is a real risk

`data_loader.py` reimplements the exact cleaning steps that `space_race_analysis.py` performs inline. Its docstring is candid about this and explains the reasoning: the static script was left "untouched" so that its behaviour could not regress, and it prints exploratory counts *before* cleaning that the dashboard does not need. The reasoning is honest and the two implementations do currently agree (verified: both yield 4,324 rows and 56 organisations).
But the README then claims the dashboard's numbers are "identical to the static charts *by construction*". That is the one word too far. Identical-by-construction is what you get when both callers run *the same code*. Here they run two copies of the same recipe, so they are identical by *coincidence and discipline*, and they will stay identical only until someone edits one copy. Note that they have already drifted stylistically: the script derives the year with `df['Date'].apply(year)`, calling a hand-written helper row by row, while the loader uses the vectorised `df['Date'].dt.year`. Same answer today, two code paths forever. Importing `load_data()` into the script and letting it do the exploratory printing separately would have cost very little.

5 The cleaning pipeline

Both pipelines perform the same five steps in the same order.

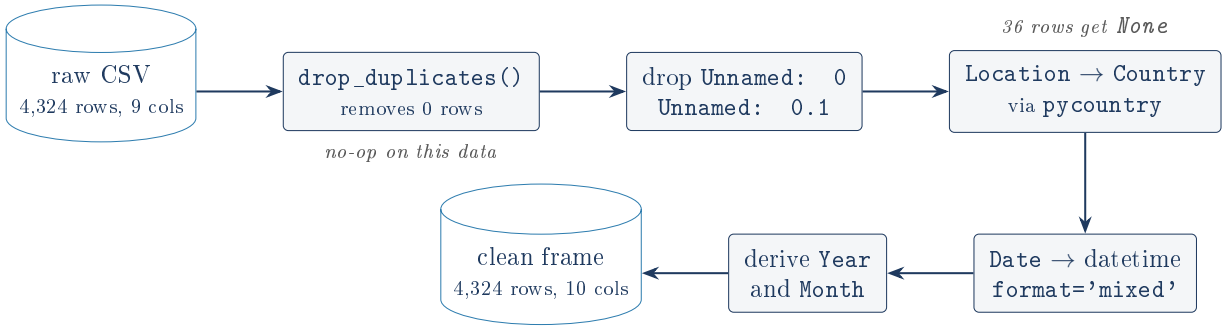

Figure 3: The cleaning pipeline. Row count never changes: no step in this pipeline removes a single row of the 4,324.

5.1 Step 1: dropping duplicates

```
1 df.drop_duplicates(inplace=True)
```

`drop_duplicates()` removes nothing. There are zero duplicates.

The README lists "drops duplicate rows" first among the project's cleaning features, and the script prints a duplicate count during exploration. Running `df.duplicated().sum()` on the raw file returns **0**. The call is a no-op on this dataset and always has been.
This is not a bug: it is defensive, it costs almost nothing, and on a re-scrape it might earn its place. But it should not be advertised as a headline feature of the cleaning, and in an interview the honest phrasing is "I check for duplicates and there are none", not "I drop duplicates". Note also that the check *could not* find a duplicate even if the same launch were listed twice, because the artefact columns are dropped *after* this line runs, and those columns hold unique row numbers. Every row is guaranteed distinct while `drop_duplicates` is looking at it. Moving the drop of the junk columns *above* the de-duplication would make the check meaningful; as ordered, it is structurally incapable of ever firing.

That last point is the sharpest thing in this section, so it is worth restating plainly: the two columns being dropped are row counters, so they are unique by definition, so every row is unique, so `drop_duplicates()` can never remove anything, regardless of the data. The step is dead code in the strict sense.

5.2 Step 2: mapping location text to a country

```python
def lookup_country(location):
    """Map a free-text 'City, Country' location string to an ISO alpha-3 code."""
    try:
        location = location.split(',')[-1].strip()
        return pycountry.countries.lookup(location).alpha_3
    except LookupError:
        return None
```

The logic: split the location on commas, take the last piece, strip the whitespace, and ask `pycountry` to resolve it. `pycountry` is a library that wraps the official ISO country register. `lookup()` is its fuzzy-ish entry point, matching against official names, common names and codes. On failure it raises `LookupError`, which the function swallows, returning `None` so one unmatched row cannot halt a 4,324-row job. Returning a sentinel instead of crashing is the right instinct.

The README's account of this function is wrong, and the truth is more interesting

The README says:

> *"A meaningful number of `Location` strings reference launch sites in the former USSR, or use informal names . . . This means the country-level charts undercount USSR-era launches."*

Measured against the actual data, this is false in both halves.
On the count: exactly 36 of 4,324 rows (0.8%) fail the lookup. Every single one of them has the same last comma-separated field, and it is not a USSR site: it is `Pacific Ocean`, the sea-launch platform. That is the complete list. There are no informal names and no unmatched Soviet sites.
On the USSR: not one Soviet launch is dropped. All 1,777 `RVSN USSR` rows resolve successfully, to `RUS` (1,198) and `KAZ` (579). The country-level charts do not undercount USSR-era launches at all.
So the stated defect does not exist. But a different and subtler one does, and the README misses it. `pycountry` only knows *present-day* countries, so Soviet launches are silently reattributed to the modern states that now occupy those coordinates. Baikonur Cosmodrome, the pad that launched Sputnik and Gagarin, is in Kazakhstan, so 579 Soviet launches are filed under `KAZ`, a country that has never had a space programme. The choropleth does not lose the USSR; it *dismembers* it, and shows a Kazakh space power that never existed. That is a far better story than the one the README tells, and it is defensible: there is no ISO code for a state that dissolved in 1991, so *some* compromise was unavoidable. The fix is a caption, not code.

5.3 Step 3: parsing the dates

```python
df['Date'] = pd.to_datetime(df['Date'], format='mixed', utc=True)
```

Jargon: Parsing a date

Converting text that a human reads as a date (`"Fri Aug 07, 2020 05:12 UTC"`) into a number the computer can compare and sort. Until you do this, "1959" and "1960" are just strings, and sorting them alphabetically only works by luck.

The `Date` column mixes formats: some rows carry a UTC suffix and some do not. Handing `pd.to_datetime` a single fixed `format` string therefore fails on whichever rows do not match it. `format='mixed'` tells pandas to infer the format *per row* instead of assuming one for the column. `utc=True` forces every result onto a single timezone so the column has one uniform type.

This is a genuinely good call and the README's reasoning for it is sound. The cost, worth knowing, is that `format='mixed'` is slow (it re-infers on every row) and is more willing to accept garbage than a strict format would be. On 4,324 rows neither matters.

5.4 Step 4: deriving Year and Month

```
MONTHS = ['January', 'February', ..., 'December']

def year(date):  return date.year
def month(date): return MONTHS[date.month - 1]

df_data['Year']  = df_data['Date'].apply(year)
df_data['Month'] = df_data['Date'].apply(month)
```

The `- 1` is the usual off-by-one bridge: months are numbered 1 to 12 by humans and lists are numbered from 0 by Python. `data_loader.py` does the same job with `.dt.year` and a lambda, as noted in Section 2's honest box.

6 The static script, chart by chart

6.1 The two helpers

Every chart is saved through one of these:

```
def save_and_maybe_show_mpl(name):
    path = os.path.join(OUTPUT_DIR, f'{name}.png')
    plt.savefig(path, bbox_inches='tight')
    if SHOW_PLOTS:
        plt.show()
    plt.close()
    print(f'Saved {path}')
```

This small function is the best design decision in the repository, and the README's account of why is accurate. Three things it gets right:

1. **Save first, show second.** A script that only calls `plt.show()` produces nothing durable and cannot prove it ran. Writing the file first makes "did this work" a directory listing rather than a judgement call.
2. **Display is opt-in** via `SHOW_PLOTS=1`. On a machine with no screen, matplotlib's `show()` warns and does nothing, while Plotly's `fig.show()` raises `ValueError` outright. Gating both behind an environment variable is what lets the whole analysis run headless, which is exactly how it was verified for this document.
3. `plt.close()` releases the figure. Without it, a script drawing 15 figures accumulates all 15 in memory and matplotlib starts warning at 20.

`BASE_PATH` is derived from `os.path.dirname(__file__)`, so the CSV and the output folder resolve relative to the script's own location, not the shell's working directory. That means `python /some/long/path/space_race_analysis.py` works from anywhere. It is a small thing that a surprising amount of analysis code gets wrong.

6.2 The launch cadence, and the shape everything else hangs off

Charts 07 and 16 both count launches per year. Here is what the data actually says, plotted from the real values:

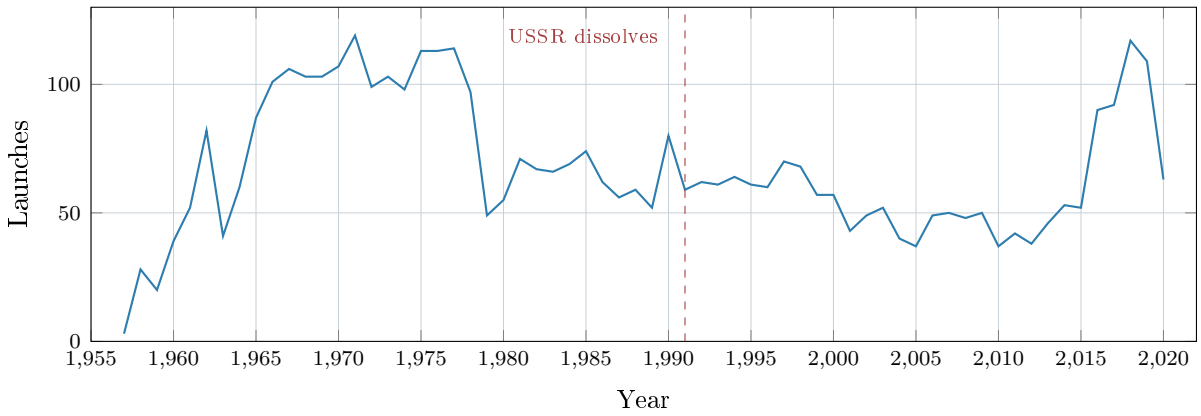

Figure 4: Launches per year, 1957 to 2020, reproduced from the project's own cleaned data. The plateau of roughly 100 launches a year through the 1970s, the collapse after 1978, the long trough of the 1990s and 2000s, and the SpaceX-era recovery after 2015 are all real features of the dataset. 2020 is low because the scrape stopped in August.

Chart 07 plots years in order of popularity, not in order

Chart 16 (`16_launches_per_year_sorted.png`) sorts by year and is the chart above. Chart 07 (`07_launches_per_year.png`) is built from a bare `df_data['Year'].value_counts()`, which pandas sorts by *count*, descending. So chart 07's x-axis runs 1971, 1977, 1975, 1976 . . . : it is the same data with the years shuffled into rank order. Its bars descend in a smooth staircase that carries no information, and because only every third tick is labelled, the axis is not even readable as a ranking.

The two charts are near-duplicates and one of them is actively misleading: a reader glancing at chart 07 sees a time axis and reads a trend that is not there. Chart 16 makes 07 redundant. Deleting 07 would improve the project.

6.3 Mission outcomes and the safety trend

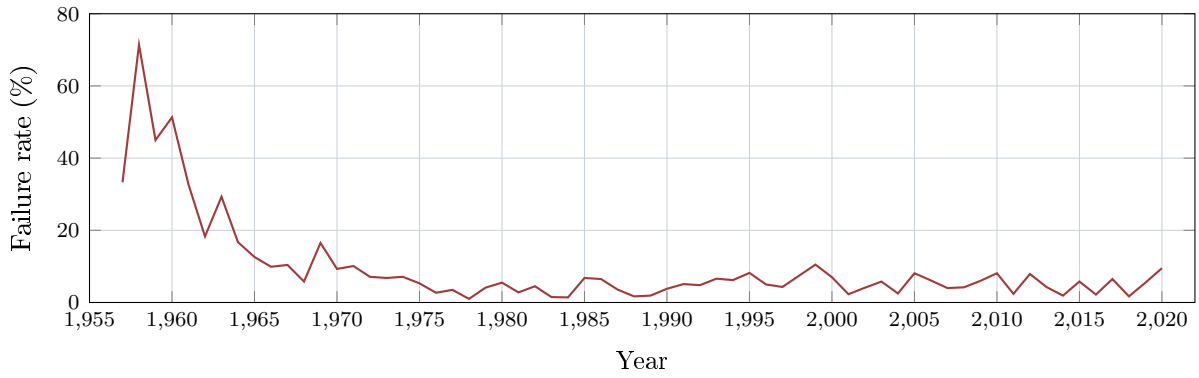

Figure 5: Percentage of launches per year with `Mission_Status == 'Failure'`, computed exactly as chart 15 computes it. The real finding of this project: rocketry went from losing half its launches to losing a few percent, and did so within about a decade.

This is the project's most defensible result. The early years are noisy because the denominators are tiny (1957 has 3 launches, so one failure is 33%), but the trend from 1958's 71.4% to a stable few-percent band by the mid-1970s is enormous and unambiguous. Chart 15 pins the y-axis to `ylim(0, 100)`, which is the correct choice: it stops the noise of the 2010s from being rescaled into looking like a crisis.

Across the whole dataset the outcome split is `Success` 3,879, `Failure` 339, `Partial Failure` 102, `Prelaunch Failure` 4, an 89.7% success rate.

The failure rate quietly ignores two of the four outcomes

Chart 15 computes `(x == 'Failure').sum() / len(x)`. That counts `Partial Failure` (102 launches) and `Prelaunch Failure` (4) as *successes*, because they are not the exact string `Failure`. Whether that is right depends on a question the project never asks: is a partial failure a failure? It is a real judgement call with a defensible answer either way, but it is currently made implicitly by a string comparison. One line of comment, or a chart that stacks all four statuses, would turn an accident into a decision.

6.4 The Cold War comparison

Charts 12 and 13 are the project's headline. Both filter to two organisations:

```
filtered_data_space_race = filtered_data[
    filtered_data['Organisation'].isin(['NASA', 'RVSN USSR'])
]
```

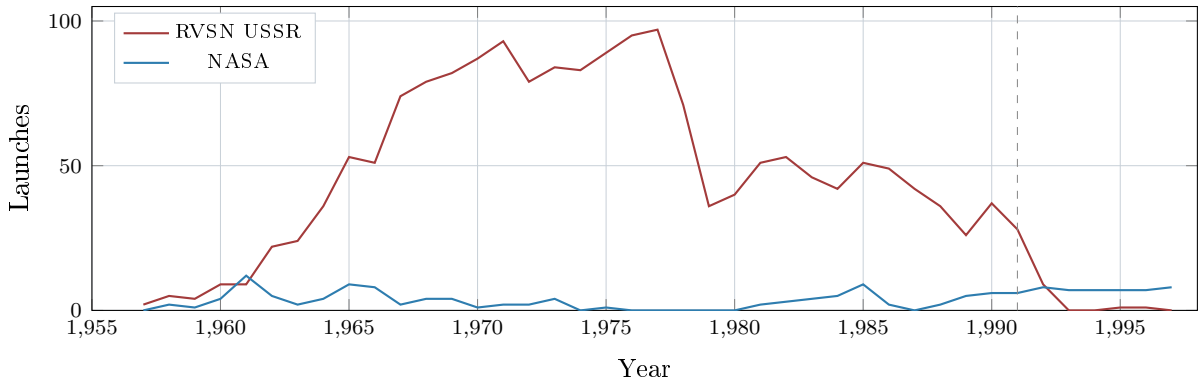

Figure 6: Chart 12 reproduced from the real data. `RVSN USSR` launches essentially stop in 1992: the organisation ceased to exist, and its successor `VKS RF` (201 launches) carries on under a different name. NASA never exceeds 12 launches in a year.

Look hard at the blue line. NASA peaks at 12 launches in 1961 and spends most of the Space Race in single digits. That should be startling: NASA put men on the Moon during the flat part of that line. The reason it is so low is the subject of the next box, and it is the most important thing in this document.

"USA vs USSR" is not comparing the USA to the USSR

`NASA` is one American organisation among many, and it is not even the largest. Of the 1,351 launches in the dataset that occurred on American soil, NASA flew **203**, which is 15.0%. The rest belong to General Dynamics (251), the US Air Force (161), ULA (140), Boeing (136), Martin Marietta (114), SpaceX (100), Northrop (83) and others. American spaceflight was overwhelmingly flown by the military and by contractors; NASA is the famous name, not the busy one.

`RVSN USSR`, by contrast, translates as the Strategic Rocket Forces, which really did run essentially the entire Soviet launch programme. So the comparison sets a 15% slice of one side against 100% of the other, and reports the result as a national scoreboard.

The damage is measurable. Chart 13's pie, over all years, reads **RVSN USSR 89.7%, NASA 10.3%**. Restricted to the Cold War era the pie logic gives NASA **5.9%** (111 launches against 1,765). But counted honestly, by launch country over the same era, it is **USA 662 against RUS+KAZ 1,770**, which is **USA 27.2%**. The chart understates the American share by a factor of about 4.6.

The USSR did out-launch the USA by roughly two and a half to one, which is a genuine and interesting finding. The chart claims nine to one. And chart 13 compounds it by running over *all years to 2020*, thirty years after the USSR ceased to exist, so a third of NASA's line and none of its rival's is drawn from a period when the race was over.

The fix is small: the `Country` column already exists, and chart 13 could compare `USA` against `RUS` plus `KAZ` within `Year <= 1991`. That the project derives exactly the column it needs and then does not use it here is the frustrating part. This is the first thing to correct, and until it is, it is the first thing to volunteer in an interview.

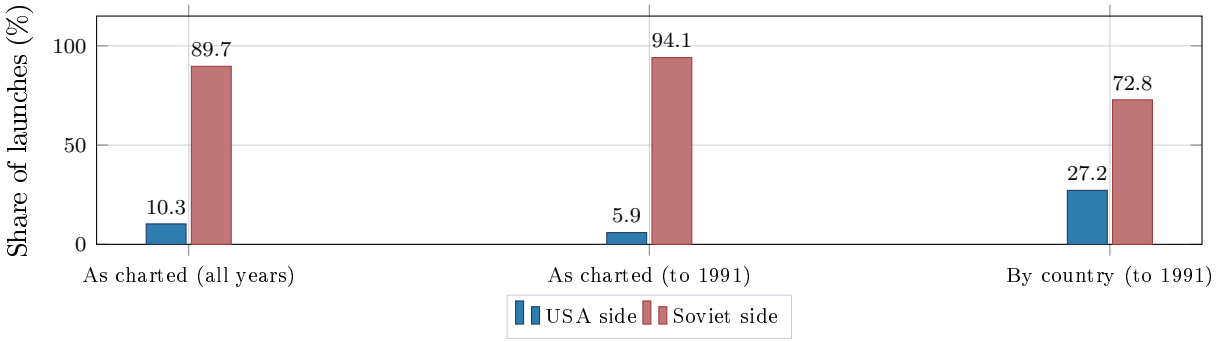

Figure 7: The size of the distortion in chart 13. The first two columns use NASA as a stand-in for the USA, as the code does. The third uses the `Country` column the project already derives. All six numbers were computed from the project's own cleaned dataframe.

6.5 Price

Only 22% of launches have a price

964 of 4,324 rows carry a `Price`; the other 3,360 are `NaN`. Every price chart calls `dropna(subset=['Price'])` first. This is correct and the README's reasoning is exactly right: treating missing as zero would invent free rockets and would make the organisations that simply do not publish costs look cheap. The consequence to keep in mind is that every price conclusion in this project describes the 22% who disclose, not the industry.

Chart 04 is a histogram of that column.

Jargon: Histogram

A chart that shows a distribution. It chops the value range into equal-width buckets ("bins") and draws a bar per bucket whose height is how many records landed in it. The number of bins is a judgement call: too few hides structure, too many turns it into noise.

```
histo_price['Price'].plot(kind='hist', bins=20)
plt.xlim(0, 1000)
```

The price histogram has 20 bins and shows two bars

These two lines fight each other. `bins=20` spreads 20 equal buckets across the *full* data range, and prices run from 5.3 to 5,000 (the Space Shuttle). So each bin is about 250 units wide. Then `xlim(0, 1000)` crops the view to the first four bins, of which only two are non-empty. The saved PNG was rendered and read for this document: it shows exactly two bars, one at 802 and one at 147, and roughly 60% of the canvas is empty whitespace to the right.

The distribution is real and interesting: median price is 62, and 98.4% of priced launches cost 500 or less. But a two-bar chart cannot show that. The single 5,000 outlier is dictating the bin width for all 964 rows, and then the crop hides the outlier that caused the problem, so the reader cannot even see why the chart looks like that.

The fix is to bin the data you intend to display, not the data you have: `data[data['Price'] <= 500].plot(kind='hist', bins=25)`, which covers 98.4% of priced launches at a 20-wide resolution. Figure 8 shows both.

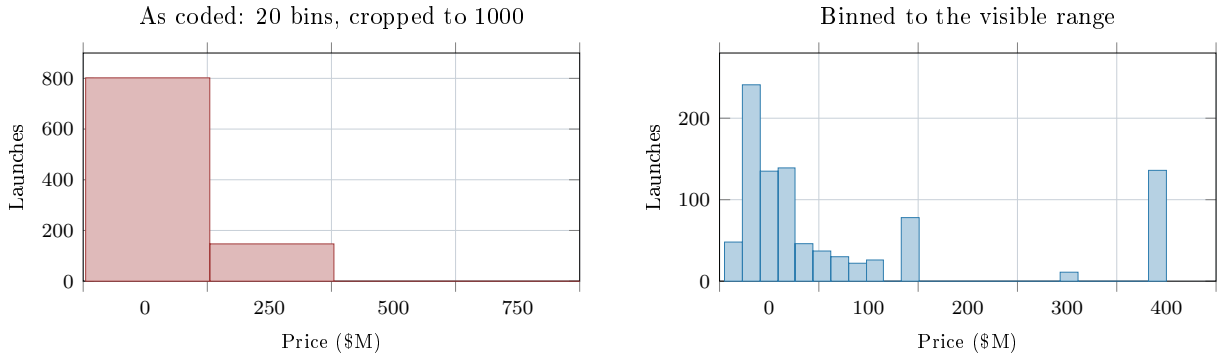

Figure 8: Left: the histogram the code actually produces, two bars and a lot of nothing. Right: the same 964 prices, binned across the range being shown. The right-hand plot reveals a lumpy, quantised distribution (many launches share an identical list price), which the left-hand plot cannot express at all. Both are computed from the real price column.

Chart 09 averages price by year.

Chart 09 draws a straight line across eight years that have no data

`filtered_data.groupby('Year')['Price'].mean().plot(kind='line')` produces a Series indexed by *only the years that have at least one price*. The priced years are 1964 to 1973, then nothing at all until 1981, then 1981 to 2020. pandas plots against the index values, so the line runs in one straight segment from 1973 to 1981, crossing eight years for which the dataset contains not a single price. A reader sees a smooth decline through the late 1970s. There was no data there to decline.

Reindexing onto the full year range would insert `NaN`s, and matplotlib draws gaps at `NaN` rather than interpolating, which is the honest rendering. The chart also has a spike to 1,687 in 1987 driven by a handful of Shuttle flights, on a y-axis shared with years averaging 60, so the entire post-1990 half of the chart is squashed flat against the bottom.

6.6 The spend table

```
money_spent_by_organizations['Total_Launches'] = \
    money_spent_by_organizations['Organisation'].map(
        df_data['Organisation'].value_counts()
    )
```

This line is the resolution of the bug the README describes at length, and the README's account of it is accurate and worth keeping. The original code indexed straight into the result of `reset_index()`, like this:

```
df['Organisation'].value_counts().reset_index()['Organisation']
```

It expected the counts, and got the organisation *names* instead, because current pandas names the columns `['Organisation', 'count']` after `reset_index()`. Strings went into a column meant for numbers, and nothing complained until a later division raised this:

```
TypeError: unsupported operand type(s) for /: 'float' and 'str'
```

Why this bug is the most instructive thing in the project

It failed *late* and *far* from its cause. The wrong data was written at one line and the exception surfaced at a completely different one, several steps later, with a message about types that says nothing about column naming. Had the intermediate column never been divided, the pipeline would have completed and produced a confidently wrong table.

`.map(series)` fixes it by matching on *keys* instead of on positions or column names: it looks each organisation up in the counts by name. That is correct regardless of what any pandas version decides to call its output columns. The general lesson, "join on identity, not on layout", is worth more than the specific fix.

The filter that follows keeps organisations with a non-zero launch count and a non-zero total price. Because organisations with no priced launches sum to 0.0, this reduces 56 organisations to **25**, matching the README's claim exactly.

`sum()` of all-missing is 0.0, and the filter depends on that

`groupby().sum()` in pandas treats `NaN` as zero, so an organisation with 31 launches and no disclosed prices gets `Price == 0.0`, not `NaN`. The filter `Price != 0.00` then removes it. The outcome is right, but only because of a default that could reasonably have gone the other way, and a float compared against zero with `!=` is a fragile idiom in general. It works here because the values are literal zeros produced by summing nothing, not the result of arithmetic that might land at `1e-17`. The table is also computed, printed to stdout, and never charted or saved, so it vanishes when the terminal scrolls.

6.7 The two Plotly charts, and a real bug in the map

Jargon: Choropleth

A map that colours each region according to a value. The colour *is* the data: darker usually means more. It only communicates if the colour is mapped to a quantity.

```
country_counts = df_data['Country'].value_counts().reset_index()
country_counts.columns = ['Country', 'Country_Appear']

fig = px.choropleth(country_counts, locations='Country', color='Country',
                    scope='world', color_continuous_scale='matter',
                    title='Number of Launches by Country',
                    labels={'Country_Appear': 'Number of Launches'},
                    hover_data=['Country_Appear'])
```

Chart 05 is titled "Number of Launches by Country" and does not show it

Read the arguments. `color='Country'` colours the map by the *country code*. The launch count lives in `Country_Appear`, which is passed only to `hover_data`. So the map is coloured by *which country a country is*, an identity, not a quantity.

Because `Country` holds strings, Plotly treats it as categorical and hands each country its own colour from a discrete palette. `color_continuous_scale='matter'` is then *silently ignored*: continuous scales do not apply to categorical data. This was confirmed by building the figure and inspecting it. The correct call produces one map trace with a colour axis; this call produces **15 separate traces**, one per country, and the figure's layout has **no colour axis at all**.

The result is a world map where Russia, the USA and Brazil are three arbitrary colours of equal visual weight. Russia has 1,398 launches and Brazil has 3. Nothing on the map says so until you hover. The chart's entire purpose is defeated by one argument.

The fix is one word: `color='Country_Appear'`. The proof that this is understood is that `app.py` *already does it right*: `build_choropleth_figure` passes `color='Launches'` and produces a single properly scaled trace. The dashboard fixed the bug and the static script was never updated to match.

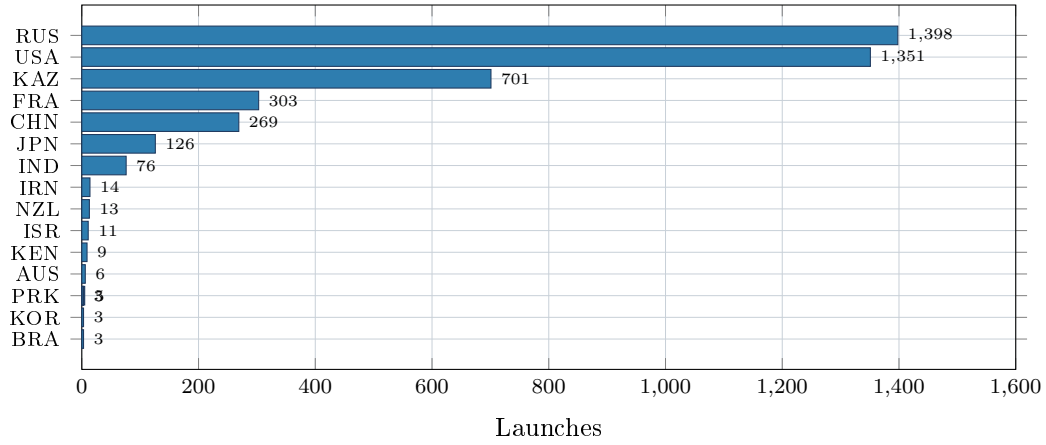

Figure 9: What chart 05 was meant to convey: the 15 countries the lookup resolves, by launch count. The 700-fold spread between `RUS` and `BRA` is precisely the information a categorical colour scheme throws away. `KAZ`'s 701 launches are Soviet flights from Baikonur, per Section 5.2.

The sunburst (chart 06) is better behaved. It nests country, then organisation, then mission status as concentric rings, and drops rows missing any of the three, which is the same 36 Pacific Ocean rows. Colouring by country is appropriate here, because a sunburst's rings encode quantity through their angular width, so colour is genuinely free to carry identity instead.

6.8 Chart 17

```
# Year-on-Year Chart Showing the Organisation Doing the Most Launches
filtered_data.groupby(['Year', 'Organisation'])['Organisation'].count() \
    .unstack().plot(kind='line', ax=plt.gca())
```

Chart 17 does not do what its comment says, and is unreadable regardless

The comment promises "the Organisation Doing the Most Launches" year on year, which would be an `idxmax` over each year: one line, or a categorical band. The code computes no maximum at all. It plots **all 56 organisations** as 56 overlapping lines.

The rendered PNG was opened and read for this document. It is worse than unreadable. The legend needs 56 entries, matplotlib places it inside the axes by default, and it grows *taller than the figure itself*: the legend box physically covers the left third of the plot, hiding the 1957 to 1975 region, which is the only part anyone wants. matplotlib also cycles its default palette every 10 colours, so the 56 lines reuse 10 colours roughly six times each and the legend cannot be used to identify anything anyway. The `Arm??e de l'Air` mojibake from Section 2.4 is sitting in that legend in plain sight.

There is also a variable-shadowing trap here. `filtered_data` is rebound four separate times in `main()`. By the time chart 17 runs, it means "rows with a non-null `Mission_Status`" (set for chart 15), which is all 4,324 rows. Chart 11 uses a *different* frame, `filtered_data_organization`, correctly restricted to the top 10. So chart 11 is the readable version of chart 17, and chart 17 is what chart 11 would have been if the filter had been forgotten. Given the comment, that is most likely what happened.

Chart 17 should either compute the actual per-year leader, or be deleted as a broken duplicate of chart 11.

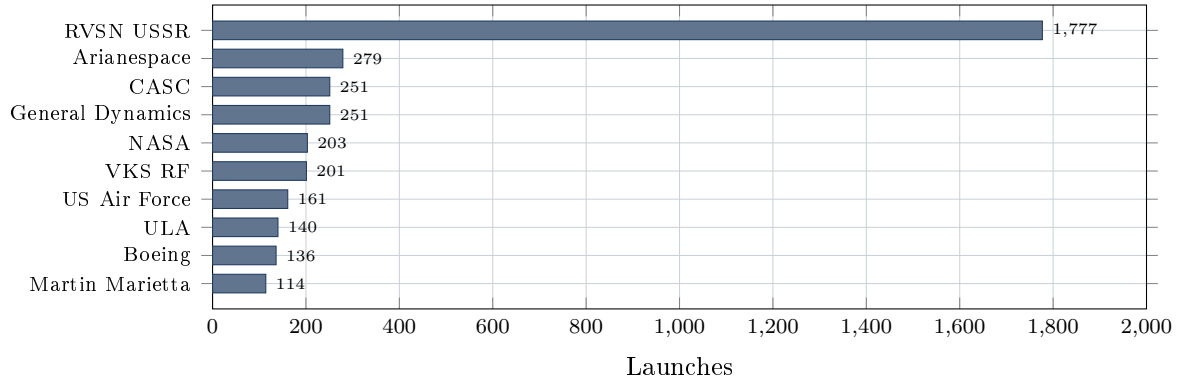

Figure 10: Chart 10 reproduced: the top 10 organisations by launch count. `RVSN USSR`'s 1,777 launches are 6.4 times the runner-up and 41% of the entire dataset. This single fact drives most of the project's findings, including the misleading pie in Section 6.4.

7 The interactive dashboard

7.1 Shape

`app.py` loads the cleaned frame *once*, at import time, into a module-level global `DF`, and every request slices that in-memory frame. Nothing is precomputed per filter combination and nothing is cached, so a filtered view is always a genuine `pandas` slice of the real data. With 4,324 rows this is instant and the design is right: the dataset is far too small to justify a database or a cache.

```
DF = load_data()

ALL_ORGANISATIONS = sorted(DF['Organisation'].dropna().unique().tolist())
YEAR_MIN = int(DF['Year'].min())
YEAR_MAX = int(DF['Year'].max())
TOP_10_ORGANISATIONS = DF['Organisation'].value_counts().head(10).index.tolist()
```

The filter option lists and the year bounds are derived from the data rather than hardcoded, which means the UI cannot drift out of sync with the CSV. `TOP_10_ORGANISATIONS` is computed the same way and handed to the browser, so the "Top 10" shortcut button reflects real counts. The README makes a point of this ("computed from the real counts, not hardcoded") and it is true.

7.2 The request cycle

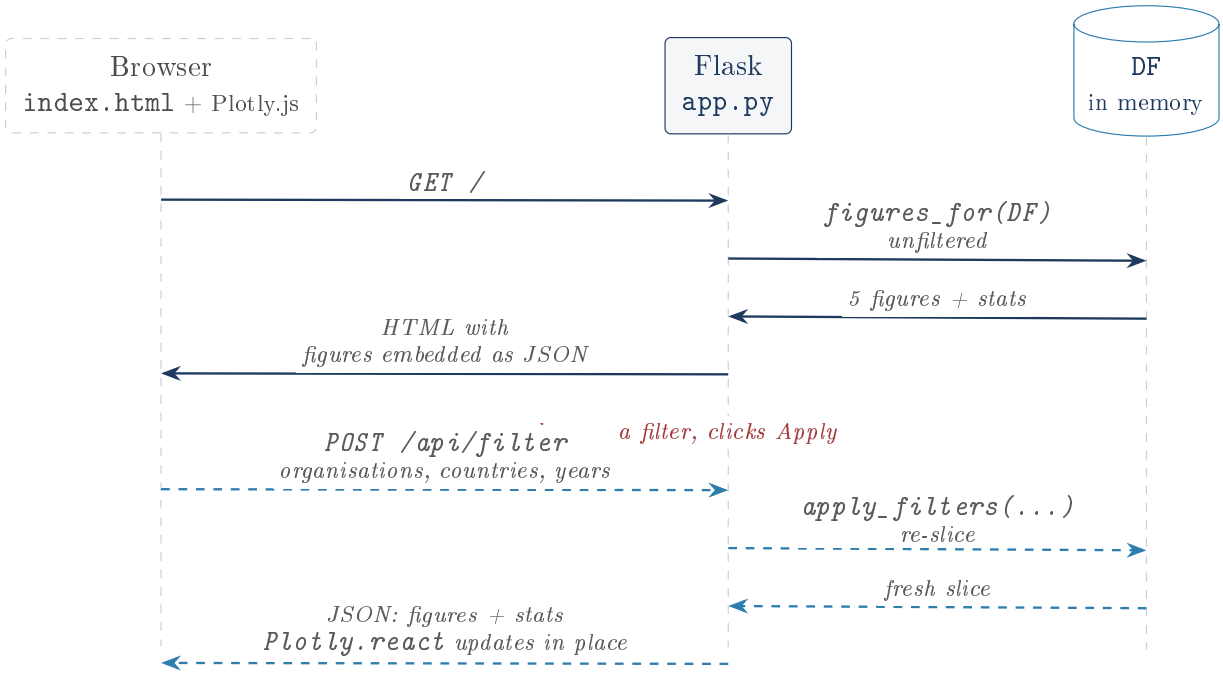

Figure 11: The dashboard's request cycle. Solid: initial page load. Dashed: every subsequent filter change, which never reloads the page.

7.3 Filtering

```
def apply_filters(organisations, countries, year_start, year_end):
    filtered = DF[(DF['Year'] >= year_start) & (DF['Year'] <= year_end)]
    if organisations:
        filtered = filtered[filtered['Organisation'].isin(organisations)]
    if countries:
        filtered = filtered[filtered['Country'].isin(countries)]
    return filtered
```

The empty-list-means-everything convention is a neat touch: an empty multi-select is "no restriction" rather than "match nothing", which is what a user expects and what the UI labels promise ("none selected = all"). Note that the year filter always applies, while the other two are conditional.

7.4 Serialising figures to the browser

```
'org_bar': json.loads(json.dumps(build_org_bar_figure(filtered),
                                 cls=plotly.utils.PlotlyJSONEncoder)),
```

Why the odd `json.loads(json.dumps(...))` round trip

A Plotly figure contains numpy integers and arrays. Python's plain `json.dumps` does not know how to serialise those and raises `TypeError`. `PlotlyJSONEncoder` does know, so `json.dumps(fig, cls=PlotlyJSONEncoder)` yields a correct JSON *string*. But `figures_for` must return a *dictionary*, because Flask's `jsonify` will serialise the whole response itself and would otherwise embed the string as a quoted string. So the code parses its own output straight back into plain Python types that `jsonify` can handle. It looks wasteful and it is: the round trip is a type laundering step, converting numpy types to native ones via JSON. It is a common Plotly-

plus-Flask idiom and the README's account of why `fig.to_html()` was unsuitable is correct: that produces a whole HTML document, not something `Plotly.react()` can consume.

Every filter change rebuilds and ships all five figures, including the map

`figures_for` always constructs all five. Changing only the year range re-serialises the choropleth and the sunburst too, and the sunburst's JSON contains one entry per country/organisation/status combination. Every response is therefore hundreds of kilobytes regardless of how little changed. At 4,324 rows on localhost this is invisible, and optimising it would be premature. It is worth knowing the ceiling exists rather than discovering it later.

7.5 Error handling

```
try:
    year_start = int(payload.get('year_start', YEAR_MIN))
    year_end = int(payload.get('year_end', YEAR_MAX))
except (TypeError, ValueError):
    return jsonify({'error': 'year_start/year_end must be integers'}), 400
```

Non-integer years return HTTP 400 with a message, verified by sending `{"year_start":"abc"}` and receiving 400. `build_stats` and both Plotly builders guard the empty-selection case explicitly, returning zeroed stats and empty figures rather than raising `IndexError` on `top_counts.index[0]`. Sending a nonexistent organisation was verified to return 200 with empty figures rather than a crash. This is more care than a portfolio dashboard usually gets.

The filters are not validated beyond the year type check

`year_start > year_end` is accepted and yields an empty selection rather than an error. Organisation and country values are passed straight into `isin()` without being checked against the known lists. That is harmless here (`isin` against junk simply matches nothing, and there is no database or shell to inject into), but the endpoint trusts its input more than it says it does.

7.6 The page

`templates/index.html` is a single self-contained file: CSS, markup and JavaScript, no build step. It uses CSS custom properties with a `prefers-color-scheme` block for dark mode, a CSS grid that collapses to one column under 800 pixels, and about 80 lines of vanilla JavaScript. `Plotly.newPlot` runs once per chart on load; `Plotly.react` runs on every update, which diffs against the current state and animates only what changed instead of tearing the chart down and rebuilding it.

The dashboard requires internet access to render anything

```
<script src="https://cdn.plot.ly/plotly-2.35.2.min.js"></script>
```

Plotly.js is loaded from a public CDN. Offline, or if that CDN is unreachable, every `Plotly.newPlot` call raises `ReferenceError: Plotly is not defined` and the page renders as a set of empty boxes with working filter controls and no charts. There is no fallback and no error message. For a project whose entire selling point is running locally against a committed CSV, an internet dependency for the rendering layer is an odd seam. Pinning the exact version (`2.35.2`) is good practice and mitigates the supply-chain half of the concern, but not the availability half.

Two template values are hardcoded and one is a latent inconsistency

`index.html` hardcodes the strings "4,324 launches, 56 organisations" in its subtitle, and hardcodes `1991` as the Cold War preset's end year. The counts are correct today, but they are computed and passed into the template for the stat tiles right below, so the subtitle will silently lie the moment the CSV changes while the tile beneath it tells the truth. `{{ stats.total_launches }}` was already available.

8 Dependencies

Package	Pinned	Role
`pandas`	`>=2.2,<4.0`	The DataFrame. Loading, cleaning, grouping, counting.
`matplotlib`	`>=3.8,<4.0`	The 15 static PNGs.
`plotly`	`>=5.20,<7.0`	The 2 static HTMLs and all 5 dashboard figures.
`pycountry`	`>=23.12,<27.0`	ISO country register, for `lookup_country`.
`Flask`	`>=3.0,<4.0`	The dashboard's web server.

Table 2: All five dependencies. Every one is used; there is nothing vestigial.

The ranges use a compatible-lower-bound, exclusive-upper-major pattern, which is the right default: it accepts patches and minors while refusing the next major, where the breaking changes live. Given that this project was bitten by exactly such a change (the `value_counts().reset_index()` naming shift), that caution is earned. Verified: the five resolve and install cleanly into a fresh virtual environment, which produced pandas 3.0.3, plotly 6.9.0 and pycountry 26.2.16, and the project runs correctly on all three.

The pin range does not actually protect against the bug that motivated it

The `reset_index()` naming change that cost the author a debugging session shipped in a pandas *minor* release, not a major one. `pandas>=2.2,<4.0` would not have stopped it. This is not an argument for pinning harder (that has its own costs), but it is worth knowing that the mitigation and the incident do not line up, and the real fix was the code change to `.map()`, which is version-independent by construction.

9 Verification log

Nothing in this document is taken on trust from the README. Everything below was run.

Claim	Source	Result
4,324 launches after cleaning	README	**confirmed**
56 organisations	README	**confirmed**
964 priced launches, about 22%	README	**confirmed** (22.3%)
25 organisations in the spend table	README	**confirmed**
17 chart files written to `output/`	README	**confirmed**, exit code 0
Cold War filter gives 1,876 launches	README	**confirmed**
111 NASA, 1,765 RVSN USSR	README	**confirmed**, via the live API
Dashboard serves `GET /`	README	**confirmed**, HTTP 200
Bad year input rejected	code	**confirmed**, HTTP 400
Unknown organisation filter	code	**confirmed**, 200, empty figures
"drops duplicate rows"	README	**no-op**: 0 duplicates exist
"undercounts USSR-era launches"	README	**false**: 0 Soviet rows dropped
36 unmatched countries	code	all are `Pacific Ocean`
"the CSV happens to be clean"	README	**false**: mojibake in 341 values
Chart 05 colours by launch count	title	**false**: 15 categorical traces
Chart 13 compares USA to USSR	title	**false**: NASA is 15% of USA
Chart 17 shows the yearly leader	comment	**false**: plots all 56 orgs

Table 3: Verification results. The upper block confirms the README; the lower block is where the code, the data or the titles disagree with what is claimed.

Method: a clean virtual environment was built from `requirements.txt`; the analysis script was run to completion (exit 0, 17 files); the resulting PNGs were compared against the committed ones (byte-identical, so the analysis is deterministic and the committed sample is a true record of the code); `app.py` was started and both of its routes were exercised with `curl`; and the cleaning was replicated independently in a scratch script to check every count. The repository was returned to a clean git state afterwards.

The committed output is reproducible

Re-running the script regenerated all 15 PNGs byte-for-byte identically to the ones in the repository. Only the two Plotly HTML files differed, and only in the embedded library version string. That is a stronger reproducibility guarantee than most analysis projects can offer, and it is worth saying out loud: the charts in `output/` are exactly what this code produces.

10 What is inferred rather than confirmed

Everything above is drawn from the source or from a measurement, except these, which are labelled inference:

- **The origin of the doubled `Unnamed` columns** (Section 2.3) is inferred from the header's shape. That the file was written, re-read and re-written by an index-emitting tool is the natural explanation, but the scrape is not in this repository and cannot be checked.
- **That chart 17 is a copy-paste of chart 11 with the filter omitted** is inferred from the comment describing something the code does not do and from chart 11 sitting six lines above with the correct frame. Plausible, not proven.
- **That the 5,000 price outlier is the Space Shuttle** is inferred from the value and the era, matching the `Detail` column's Shuttle rows. The check was not run exhaustively.
- **That `RVSN USSR` covers essentially the whole Soviet programme**, unlike NASA on the American side, is domain knowledge, not something the dataset states. The dataset does show that no other Soviet organisation appears in comparable numbers, which is consistent with it.

11 Defending this project in an interview

Lead with the bugs

This project's strengths are real: the headless-first chart saving, the key-based `.map()` fix, the deliberate `dropna` on price, the derived-not-hardcoded UI options, the reproducible committed output. But its titles overclaim in four places, and every one of them is findable in under a minute by anyone who reads the code. Volunteering them converts each from a liability into evidence of judgement. The order that costs least and demonstrates most:

1. **"My USA vs USSR chart is wrong, and here is the real number."** NASA is 15% of American launches, so the chart says 9:1 when the truth is about 2.5:1. The `Country` column needed to fix it already exists. This is the single most important thing to say, and saying it first defuses everything else.
2. **"The choropleth is coloured by country, not by count."** One argument. `app.py` already has it right, which shows the fix is understood.
3. **"The price histogram shows two bars because one 5,000 outlier sets the bin width."** Then describe the real distribution: median 62, and 98.4% under 500.
4. **"My duplicate check can never fire"**, because the row-number columns are dropped after it runs. That is a nice thing to have noticed about your own code.

Then the correction to the README's own caveat, which is the best story here: the country lookup does not lose the USSR, it *reattributes* it, and files 579 Soviet launches from Baikonur under Kazakhstan. Knowing precisely how your data is wrong, rather than vaguely that it might be, is the whole job.